POSTECH FIAP

Arquitetura e Desenvolvimento Java

RELATORIO TÉCNICO

Tech Challenge - Fase 01

Sistema de Gestão de Usuários para Restaurantes

Alunos	**RM370614 - Vinicius Campos Fanti** **RM373960 - Alex Sousa** **RM373739 - Lucas Gonçalves Prado das Neves** **RM372334 - Higo Otaviano da Rocha** **RM371904 - Marcos Sanches Polido Junior**
Curso	Arquitetura e Desenvolvimento Java
Tecnologia	Spring Boot 4.0.4 + PostgreSQL 17 + Docker
Data	05/04/2026

1. Descrição da Arquitetura

A aplicação foi desenvolvida utilizando Spring Boot 4.0.4 seguindo o padrão de arquitetura em camadas (Layered Architecture), garantindo separação de responsabilidades, testabilidade e manutenibilidade do código. Cada camada possui uma responsabilidade única e bem definida, seguindo os princípios SOLID e boas práticas de Orientação a Objetos.

Diagrama de Camadas

Cliente / Postman / Swagger UI
Controller — Recebe requisicoes HTTP, valida entrada com @Valid
Service — Regras de negocio, validacoes e orquestracao
Repository — Acesso ao banco via Spring Data JPA
PostgreSQL 17 — Banco de Dados Relacional (Docker)

Responsabilidade de cada camada

Camada	Classe	Responsabilidade
Controller	`UserController`	Recebe requisições HTTP, válida entrada e delega ao Service
Service	`UserService`	Regras de negocio: e-mail unico, validação de senha, tipo de usuário
Repository	`UserRepository`	Interface JPA com Query Methods para acesso ao banco
Model	`User, Customer, RestaurantOwner`	Entidades JPA com heranca JOINED e Address @Embeddable
DTO	`CreateUserRequest, UserResponse, etc.`	Protege dados sensíveis e desacopla API do modelo interno
Exception	`GlobalExceptionHandler`	Intercepta exceções e retorna ProblemDetail (RFC 7807)

2. Modelagem das Entidades e Relacionamentos

O sistema utiliza herança de entidades JPA com a estratégia JOINED, onde os campos comuns ficam na tabela users e cada tipo de usuário possui sua própria tabela com chave estrangeira referenciando a tabela pai. O endereço foi modelado como @Embeddable, incorporando seus campos diretamente na tabela users.

Hierarquia de Classes

```
User (abstract)  ←  campos comuns: id, name, email, login, password, lastModifiedAt, address
  ├── Customer         → tabela: customers    (userType: CUSTOMER)
  └── RestaurantOwner  → tabela: restaurant_owners  (userType: RESTAURANT_OWNER)
```

Tipos de Usuario

Tipo Java	Tabela	userType no Request	Descricao
Customer	`customers`	**CUSTOMER**	Cliente que faz pedidos
RestaurantOwner	`restaurant_owners`	**RESTAURANT_OWNER**	Dono de restaurante

Campos Obrigatórios (Tabela users)

Campo Java	Coluna SQL	Tipo	Restrições
`id`	`id`	BIGINT	**PK, AUTO_INCREMENT**
`name`	`name`	VARCHAR(255)	NOT NULL
`email`	`email`	VARCHAR(255)	**NOT NULL, UNIQUE**
`login`	`login`	VARCHAR(255)	NOT NULL
`password`	`password`	VARCHAR(255)	NOT NULL
`lastModifiedAt`	`last_modified_at`	TIMESTAMP	Auto-atualizado
`address.street`	`street`	VARCHAR(255)	@Embeddable
`address.city`	`city`	VARCHAR(255)	@Embeddable
`address.zipCode`	`zip_code`	VARCHAR(255)	@Embeddable

3. Endpoints Disponives

Todos os endpoints seguem o padrão REST e estão versionados sob o prefixo **/api/v1/users**. O versionamento garante compatibilidade retroativa quando novas versões forem introduzidas em fases futuras.

Metodo	URL	Status	Descricao
POST	`/api/v1/users`	201	Cadastrar usuário (CUSTOMER ou RESTAURANT_OWNER)
GET	`/api/v1/users/{id}`	200	Buscar usuario por ID
GET	`/api/v1/users/search?name=`	200	Buscar usuários por nome (case-insensitive)
PUT	`/api/v1/users/{id}`	200	Atualizar dados gerais (sem senha)
PATCH	`/api/v1/users/{id}/password`	204	Trocar senha (endpoint exclusivo)
DELETE	`/api/v1/users/{id}`	204	Excluir usuario
POST	`/api/v1/users/login`	200	Validar login e senha

Exemplo 1 — POST /api/v1/users (Cadastrar Usuário)

Requisicao:

```
{
  "name": "Joao Silva",
  "email": "joao@email.com",
  "login": "joao123",
  "password": "senha123",
  "userType": "CUSTOMER",
  "address": {
    "street": "Rua das Flores",
    "number": "123",
    "city": "Sao Paulo",
    "zipCode": "01310-100"
  }
}
```

Resposta de sucesso (201 Created):

```
{
  "id": 1,
  "name": "Joao Silva",
  "email": "joao@email.com",
  "login": "joao123",
  "userType": "Customer",
  "address": { "street": "Rua das Flores", "number": "123", "city": "Sao Paulo", "zipCode":
"01310-100" },
  "lastModifiedAt": "2026-04-05T18:00:00"
}
```

Resposta de erro — E-mail duplicado (400 Bad Request):

```
{
  "type": "https://api.restaurant.com/errors/invalid-request",
  "title": "Requisicao invalida",
  "status": 400,
  "detail": "E-mail ja cadastrado",
  "instance": "/api/v1/users"
}
```

Resposta de erro — Campos obrigatorios faltando (422 Unprocessable Entity):

```
{
  "type": "https://api.restaurant.com/errors/validation",
  "title": "Erro de validacao",
  "status": 422,
  "detail": "email: E-mail invalido, name: Nome e obrigatorio",
  "instance": "/api/v1/users"
}
```

Exemplo 2 — GET /api/v1/users/{id} (Buscar por ID)

Requisicao: GET http://localhost:8080/api/v1/users/1

Resposta de sucesso (200 OK):

```
{
  "id": 1,
  "name": "Joao Silva",
  "email": "joao@email.com",
  "login": "joao123",
  "userType": "Customer",
  "address": { "street": "Rua das Flores", "number": "123", "city": "Sao Paulo", "zipCode":
"01310-100" },
  "lastModifiedAt": "2026-04-05T18:00:00"
}
```

Resposta de erro — ID inexistente (404 Not Found):

```
{
  "type": "https://api.restaurant.com/errors/not-found",
  "title": "Recurso nao encontrado",
  "status": 404,
  "detail": "Usuario com id 999 nao encontrado",
  "instance": "/api/v1/users/999"
}
```

Exemplo 3 — GET /api/v1/users/search?name= (Buscar por Nome)

Requisicao: GET http://localhost:8080/api/v1/users/search?name=Joao

Resposta de sucesso (200 OK) — lista com usuarios encontrados:

```
[
  {
    "id": 1,
    "name": "Joao Silva",
    "email": "joao@email.com",
    "login": "joao123",
    "userType": "Customer",
    "address": { "street": "Rua das Flores", "number": "123", "city": "Sao Paulo",
"zipCode": "01310-100" },
    "lastModifiedAt": "2026-04-05T18:00:00"
  }
]
```

Resposta quando nenhum usuario e encontrado (200 OK — lista vazia):

```
[]
```

Exemplo 4 — PUT /api/v1/users/{id} (Atualizar Dados)

Requisicao (PUT http://localhost:8080/api/v1/users/1):

```
{
  "name": "Joao Silva Atualizado",
  "email": "joao.atualizado@email.com",
  "login": "joao456",
  "address": {
    "street": "Av. Paulista",
    "number": "1000",
    "city": "Sao Paulo",
    "zipCode": "01310-200"
  }
}
```

Resposta de sucesso (200 OK):

```
{
  "id": 1,
  "name": "Joao Silva Atualizado",
  "email": "joao.atualizado@email.com",
  "login": "joao456",
  "userType": "Customer",
  "address": { "street": "Av. Paulista", "number": "1000", "city": "Sao Paulo", "zipCode":
"01310-200" },
  "lastModifiedAt": "2026-04-05T19:00:00"
}
```

Resposta de erro — Usuário não encontrado (404 Not Found):

```
{
  "type": "https://api.restaurant.com/errors/not-found",
  "title": "Recurso nao encontrado",
  "status": 404,
  "detail": "Usuario com id 999 nao encontrado",
  "instance": "/api/v1/users/999"
}
```

Exemplo 5 — PATCH /api/v1/users/{id}/password (Trocar Senha)

Requisicao (PATCH http://localhost:8080/api/v1/users/1/password):

```
{
  "currentPassword": "senha123",
  "newPassword": "novaSenha456"
}
```

Resposta de sucesso (204 No Content):

```
(sem corpo na resposta)
```

Resposta de erro — Senha atual incorreta (400 Bad Request):

```
{
  "type": "https://api.restaurant.com/errors/invalid-request",
  "title": "Requisicao invalida",
  "status": 400,
  "detail": "Senha atual incorreta",
  "instance": "/api/v1/users/1/password"
}
```

Resposta de erro — Usuario nao encontrado (404 Not Found):

```
{
  "type": "https://api.restaurant.com/errors/not-found",
  "title": "Recurso nao encontrado",
  "status": 404,
  "detail": "Usuario com id 999 nao encontrado",
  "instance": "/api/v1/users/999/password"
}
```

Exemplo 6 — DELETE /api/v1/users/{id} (Excluir Usuario)

Requisicao: DELETE http://localhost:8080/api/v1/users/1

Resposta de sucesso (204 No Content):

```
(sem corpo na resposta)
```

Resposta de erro — Usuário não encontrado (404 Not Found):

```
{
  "type": "https://api.restaurant.com/errors/not-found",
  "title": "Recurso nao encontrado",
  "status": 404,
  "detail": "Usuario com id 999 nao encontrado",
  "instance": "/api/v1/users/999"
}
```

Exemplo 7 — POST /api/v1/users/login (Validar Login)

Requisicao:

```
{
  "login": "joao123",
  "password": "senha123"
}
```

Resposta de sucesso (200 OK):

```
{
  "id": 1,
  "name": "Joao Silva",
  "email": "joao@email.com",
  "login": "joao123",
  "userType": "Customer",
  "address": { "street": "Rua das Flores", "number": "123", "city": "Sao Paulo", "zipCode":
"01310-100" },
  "lastModifiedAt": "2026-04-05T18:00:00"
}
```

Resposta de erro — Login ou senha inválidos (400 Bad Request):

```
{
  "type": "https://api.restaurant.com/errors/invalid-request",
  "title": "Requisicao invalida",
  "status": 400,
  "detail": "Login ou senha invalidos",
  "instance": "/api/v1/users/login"
}
```

4. Documentacao Swagger / OpenAPI

A documentação foi gerada automaticamente pela biblioteca **SpringDoc OpenAPI**, com enriquecimento via anotações **@Operation**, **@ApiResponses** e **@Schema**.

> Swagger UI disponível em: http://localhost:8080/swagger-ui/index.html

O que foi documentado

- Todos os 7 endpoints com @Operation (summary e description)
- Todos os status de resposta com @ApiResponses
- Respostas de erro com schema ProblemDetail (RFC 7807) e exemplos reais
- Campos dos DTOs com @Schema (description e example)
- Valores permitidos para userType: CUSTOMER e RESTAURANT_OWNER
- Agrupamento de endpoints com @Tag(name = "Usuarios")

Configuracao SwaggerConfig.java

```
@Configuration
public class SwaggerConfig {
    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Restaurant Users API")
                .version("v1")
                .description("API de gerenciamento de usuarios")
                .contact(new Contact()
                    .name("Vinicius Campos Fanti")));
    }
}
```

5. Colecao Postman

A colecção foi exportada como **restaurant-users.postman_collection.json** (Collection v2.1). Utiliza a variável de ambiente **{{base_url}}** configurada como **http://localhost:8080/api/v1**.

Cenario	Metodo	Status Esperado
Cadastro de usuario valido (CUSTOMER)	**POST**	201 Created
Cadastro de usuario valido (RESTAURANT_OWNER)	**POST**	201 Created
Cadastro invalido — e-mail duplicado	**POST**	400 Bad Request
Cadastro invalido — campos obrigatorios faltando	**POST**	422 Unprocessable
Buscar usuario por ID — sucesso	**GET**	200 OK
Buscar usuario por ID — nao encontrado	**GET**	404 Not Found
Buscar usuarios por nome	**GET**	200 OK
Atualizar dados — sucesso	**PUT**	200 OK
Atualizar dados — usuario nao encontrado	**PUT**	404 Not Found
Trocar senha — sucesso	**PATCH**	204 No Content
Trocar senha — senha atual incorreta	**PATCH**	400 Bad Request
Validar login — sucesso	**POST**	200 OK
Validar login — invalido	**POST**	400 Bad Request
Excluir usuario — sucesso	**DELETE**	204 No Content
Excluir usuario — nao encontrado	**DELETE**	404 Not Found

6. Estrutura do Banco de Dados

Banco utilizado: **PostgreSQL 17** executando em container Docker. As tabelas são criadas automaticamente pelo Hibernate com **ddl-auto: update**. A estratégia JOINED gera três tabelas relacionadas.

Tabela: users

Coluna	Tipo SQL	Restricões	Descrição
`id`	BIGINT	**PK, AUTO_INCREMENT**	Identificador unico
`name`	VARCHAR(255)	NOT NULL	Nome completo
`email`	VARCHAR(255)	**NOT NULL, UNIQUE**	E-mail unico
`login`	VARCHAR(255)	NOT NULL	Login de acesso
`password`	VARCHAR(255)	NOT NULL	Senha
`last_modified_at`	TIMESTAMP	-	Data última alteração
`street`	VARCHAR(255)	-	Rua (@Embeddable)
`number`	VARCHAR(255)	-	Numero (@Embeddable)
`city`	VARCHAR(255)	-	Cidade (@Embeddable)
`zip_code`	VARCHAR(255)	-	CEP (@Embeddable)
`dtype`	VARCHAR(31)	NOT NULL	Discriminador JPA

Tabelas Filhas (herança JOINED)

Tabela	Coluna	Tipo	Restricao
`restaurant_owners`	`id`	BIGINT	**PK, FK -> users.id**
`customers`	`id`	BIGINT	**PK, FK -> users.id**

7. Execução com Docker Compose

A aplicação e o banco de dados rodam em containers Docker orquestrados pelo Docker Compose, dentro de uma rede dedicada chamada restaurant_network. Os containers se comunicam pelo nome do serviço, não por localhost.

Variáveis de Ambiente

Variavel	Valor	Descricao
`SPRING_DATASOURCE_URL`	`jdbc:postgresql://postgres:5432/restaurant_users`	URL de conexão com banco
`SPRING_DATASOURCE_USERNAME`	`admin`	Usuario do banco
`SPRING_DATASOURCE_PASSWORD`	`admin123`	Senha do banco
`POSTGRES_DB`	`restaurant_users`	Nome do banco

Passo a Passo para Execução

#	Etapa	Comando / Acao
1	**Pre-requisito**	`Ter Docker e Docker Compose instalados`
2	**Clonar o repositorio**	`git clone <url-do-repositorio>`
3	**Entrar na pasta**	`cd restaurant-users`
4	**Subir os containers**	`docker compose up --build -d`
5	**Verificar containers**	`docker compose ps`
6	**Acessar a API**	`http://localhost:8080/api/v1/users`
7	**Acessar Swagger**	`http://localhost:8080/swagger-ui/index.html`
8	**Parar os containers**	`docker compose down`

8. Testes Unitarios (JUnit 5 + Mockito)

Foram implementados **14 testes unitários** cobrindo todos os cenários principais do **UserService**. Os testes utilizam **Mockito** para simular o comportamento do **UserRepository**, garantindo isolamento total do banco de dados. Todos os 14 testes passaram com sucesso.

Teste	Cenario	Resultado
`deveCriarUsuarioComSucesso`	Cria usuario com dados validos	**PASS**
`deveLancarExcecaoEmailDuplicado`	Rejeita e-mail ja existente	**PASS**
`deveLancarExcecaoTipoInvalido`	Rejeita userType invalido	**PASS**
`deveBuscarUsuarioPorIdComSucesso`	Retorna usuario por ID	**PASS**
`deveLancarExcecaoIdInexistente`	Lanca ResourceNotFoundException	**PASS**
`deveBuscarUsuariosPorNome`	Retorna lista por nome	**PASS**
`deveRetornarListaVaziaAoBuscarNomeInexistente`	Retorna lista vazia	**PASS**
`deveAtualizarUsuarioComSucesso`	Atualiza dados gerais	**PASS**
`deveLancarExcecaoAoAtualizarComEmailDuplicado`	Rejeita e-mail duplicado na atualizacao	**PASS**
`deveTrocarSenhaComSucesso`	Troca senha com senha atual correta	**PASS**
`deveLancarExcecaoSenhaAtualIncorreta`	Rejeita senha atual errada	**PASS**
`deveValidarLoginComSucesso`	Valida credenciais corretas	**PASS**
`deveLancarExcecaoLoginInvalido`	Rejeita senha errada no login	**PASS**
`deveDeletarUsuarioComSucesso`	Deleta usuario existente	**PASS**

14 Testes	14 Passando	0 Falhando	100% Sucesso